



UNIVERSIDAD DE LAS AMERICAS

ISWZ1103 - PROGRAMACIÓN II

EJERCICIO PRÁCTICO RC1

Informe técnico y solución funcional

Sistema de gestión de inventarios para una tienda de productos

Integrantes:	Martin Aimacaña / Alex Palma
Tema:	Tienda de productos: CRUD de productos y gestión de inventarios (comprar, vender)
Modalidad:	Grupo de 2 estudiantes
Herramientas:	Java, IntelliJ IDEA, POO, UML, Draw.io/Graphviz y GitHub
Fecha:	1 de junio de 2026

Resultado de aprendizaje RC1: Identifica, formula y resuelve problemas complejos de ingeniería mediante principios de ingeniería, ciencia y matemática.

Índice

- 1. Formulación del Problema
- 2. Propuesta de Alternativas
- 3. Selección y Justificación
- 4. Documentación y Evidencias
- 5. Casos de Prueba
- 6. Conclusión

1. Formulación del Problema

1.1 Descripción del escenario

La pequeña empresa “Tienda de Productos” realiza ventas diarias de artículos perecederos y no perecederos. Actualmente el control de inventario se realiza de forma manual, lo cual provoca errores al registrar productos, pérdidas por productos vencidos, desconocimiento del stock real, ventas sin disponibilidad suficiente y dificultad para calcular el valor económico del inventario. Por esta razón, se plantea desarrollar un sistema en Java aplicando Programación Orientada a Objetos para gestionar el CRUD de productos y las operaciones de compra y venta.

El problema principal consiste en diseñar e implementar una solución funcional que permita registrar, consultar, actualizar y eliminar productos, además de controlar entradas por compra y salidas por venta, respetando restricciones de presupuesto, capacidad de bodega y validación de datos obligatorios.

1.2 Requerimientos funcionales

Código	Requerimiento	Descripción
RF1	Registrar producto	Permitir el ingreso de productos con nombre, cantidad, precio de compra, precio de venta, stock mínimo y tipo.
RF2	Consultar inventario	Listar todos los productos registrados, mostrando stock, tipo y valor del inventario.
RF3	Actualizar producto	Modificar datos principales como nombre, precios y stock mínimo.
RF4	Eliminar producto	Eliminar un producto del inventario mediante su ID.
RF5	Comprar producto	Aumentar el stock de un producto y descontar el presupuesto disponible.
RF6	Vender producto	Reducir el stock y evitar ventas cuando no exista cantidad suficiente.
RF7	Alertas	Mostrar productos con bajo stock y productos perecederos vencidos.

1.3 Identificación de variables

Elemento	Variables	Uso dentro del sistema
Producto	id, nombre, tipo	Identifica cada artículo de forma única.
Stock	cantidad, stockMinimo	Controla disponibilidad y alerta de reposición.
Costos	precioCompra, precioVenta	Permite calcular valor de inventario y ganancia.
Fechas	fechaRegistro, fechaVencimiento	Sirve para controlar productos perecederos.
Espacio	espacioUnidad, capacidadBodega	Evita superar la capacidad del local o bodega.
Presupuesto	presupuestoDisponible	Limita las compras según el dinero disponible.

1.4 Restricciones del sistema

- Presupuesto inicial de compra: \$500,00. El sistema no permite comprar si el costo supera este valor disponible.
- Capacidad máxima de bodega: 200 unidades de espacio. Cada producto ocupa un espacio por unidad.
- Los campos obligatorios no pueden estar vacíos, especialmente nombre, cantidad, precios y tipo de producto.
- La cantidad, el stock mínimo y las unidades de compra o venta no pueden ser negativas.
- El precio de compra y el precio de venta deben ser mayores que cero.
- No se permite vender más unidades que el stock disponible.
- Los productos perecederos requieren fecha de vencimiento para detectar productos vencidos.

2. Propuesta de Alternativas

Alternativa 1: Sistema básico con una sola clase Producto

Esta alternativa propone crear una clase Producto con todos los atributos posibles, incluyendo fecha de vencimiento y categoría, aunque no todos los productos necesiten esos datos. La clase Inventario gestionaría el registro, listado, actualización, eliminación, compra y venta.

- Encapsulamiento: los atributos del producto se mantienen privados y se acceden por métodos.
- Herencia: no se aplica o se aplica de forma limitada, porque todo se concentra en una sola clase.
- Polimorfismo: no se aprovecha correctamente, ya que todos los productos se tratan igual aunque tengan comportamientos distintos.
- Ventaja: implementación rápida y sencilla.
- Desventaja: poca escalabilidad, código menos mantenible y datos innecesarios para productos no perecederos.

Alternativa 2: Sistema POO con jerarquía de productos

Esta alternativa propone una clase abstracta Producto y dos clases hijas: ProductoPerecedero y ProductoNoPerecedero. La clase Inventario se encarga de la lógica de negocio y la interfaz Main permite interactuar mediante consola. Esta opción organiza el código en paquetes modelo, negocio e interfaz.

- Encapsulamiento: los atributos son privados y se controlan mediante métodos de acceso y validaciones.
- Herencia: ProductoPerecedero y ProductoNoPerecedero heredan atributos y comportamientos comunes de Producto.
- Polimorfismo: métodos como estaVencido() y getTipo() se comportan de forma diferente según el tipo de producto.
- Ventaja: solución organizada, extensible, alineada con POO y adecuada para agregar nuevos tipos de productos.
- Desventaja: requiere más clases y un diseño inicial más detallado.

Criterio	Alternativa 1	Alternativa 2
Uso de POO	Básico	Completo: encapsulamiento, herencia y polimorfismo
Mantenibilidad	Media	Alta, porque separa responsabilidades
Escalabilidad	Baja	Alta, permite nuevos tipos de productos
Control de vencimiento	Limitado	Especializado para productos perecederos
Alineación con la consigna	Parcial	Alta

3. Selección y Justificación

Se selecciona la Alternativa 2 porque ofrece una solución integral y técnicamente más sólida para el problema formulado. Esta opción usa correctamente los principios de Programación Orientada a Objetos, mantiene las responsabilidades separadas y permite ampliar el sistema en el futuro sin modificar de manera excesiva las clases existentes.

- La clase abstracta Producto centraliza los atributos comunes y evita duplicación de código.
- Las subclases ProductoPerecedero y ProductoNoPerecedero permiten diferenciar comportamientos mediante polimorfismo.
- La clase Inventario concentra la lógica de negocio: CRUD, compras, ventas, cálculo de inventario y alertas.

- El paquete interfaz separa la interacción con el usuario de la lógica interna del sistema.
- La validación de datos reduce errores de ejecución y evita registros incompletos o ventas imposibles.

3.1 Diagrama UML de clases

El siguiente diagrama representa la estructura seleccionada del sistema:

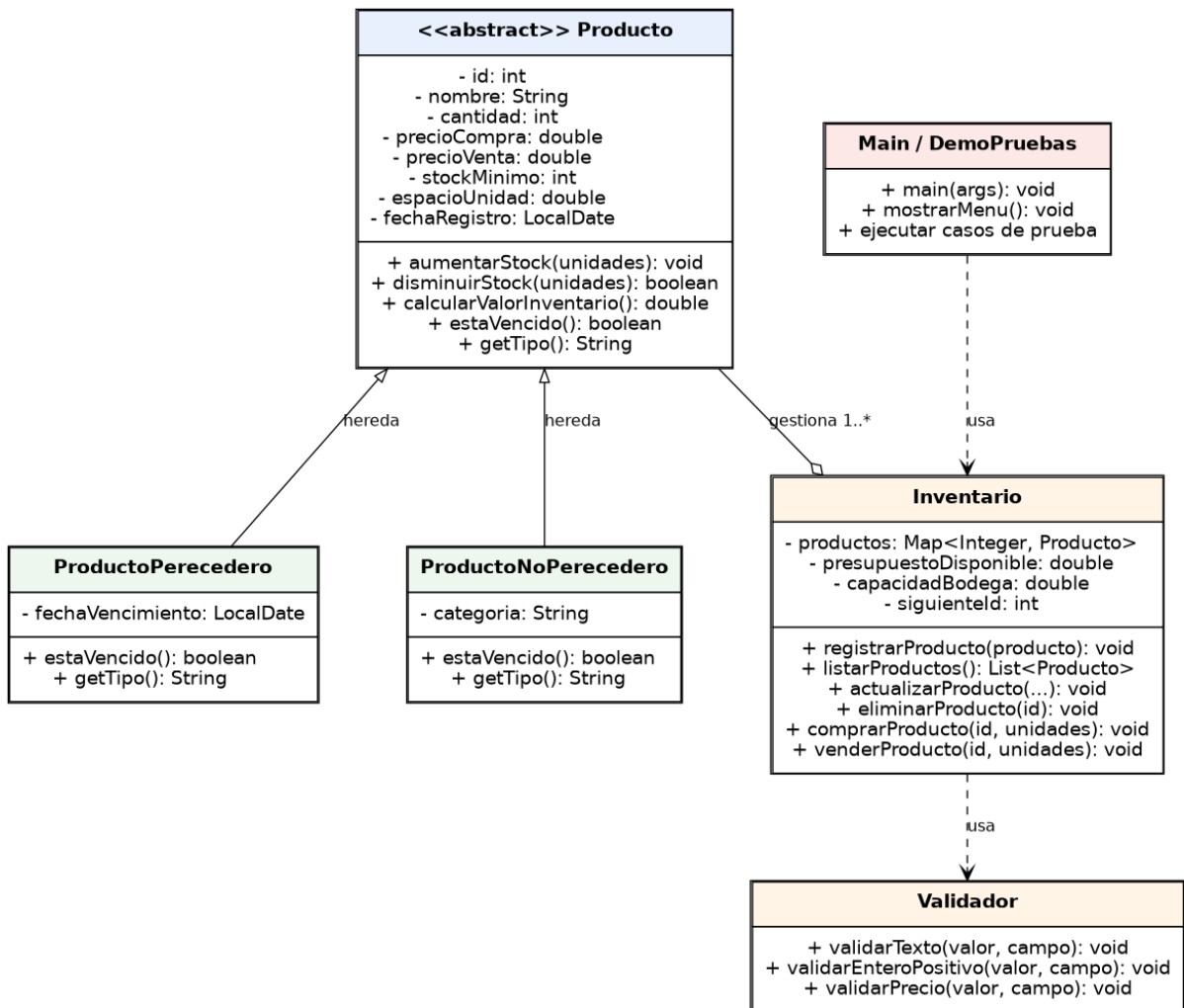


Figura 1. Diagrama UML de clases del sistema de inventario.

4. Documentación y Evidencias

4.1 Enlace público al repositorio

Enlace de GitHub: [REEMPLAZAR AQUÍ POR EL ENLACE PÚBLICO DEL REPOSITORIO AL SUBIR EL PROYECTO]

Repositorio local entregado: carpeta TiendaProductosPOO, organizada con paquetes modelo, negocio e interfaz. El README y la ejecución en IntelliJ IDEA identifican al grupo conformado por Martin Aimacaña y Alex Palma.

4.2 Organización del código fuente

Clase	Función principal
modelo.Producto	Clase abstracta base. Contiene atributos comunes y métodos para aumentar/disminuir stock, calcular valor de inventario y definir métodos polimórficos.
modelo.ProductoPerecedero	Clase hija para productos con fecha de vencimiento. Sobrescribe estaVencido() y getTipo().
modelo.ProductoNoPerecedero	Clase hija para productos sin vencimiento cercano. Maneja categoría y sobrescribe métodos polimórficos.
negocio.Inventario	Gestiona el CRUD de productos, compras, ventas, alertas, presupuesto y capacidad de bodega.
negocio.Validador	Centraliza validaciones de texto, cantidades y precios.
negocio.ValidacionException	Permite mostrar errores controlados y claros al usuario.
interfaz.Main	Menú de consola para usar el sistema de forma interactiva. Al ejecutar en IntelliJ IDEA muestra el encabezado del grupo.
interfaz.DemoPruebas	Ejecuta casos de prueba automáticos para generar evidencias e incluye los integrantes del grupo en la salida.

4.3 Lógica general del código

- El usuario registra productos desde la interfaz. Según el tipo seleccionado, se crea un objeto ProductoPerecedero o ProductoNoPerecedero.
- Inventario valida los datos antes de guardar el producto, evitando campos vacíos, precios inválidos o cantidades negativas.
- Al comprar, el sistema aumenta el stock, descuenta el presupuesto y verifica capacidad de bodega.
- Al vender, el sistema reduce el stock únicamente si existen unidades suficientes.
- Las alertas se obtienen consultando productos con cantidad menor o igual al stock mínimo y productos perecederos vencidos.

5. Casos de Prueba

CP1: Validación de registro de producto con campos vacíos

Elemento	Detalle
ID y Nombre	CP1: Validación de registro de producto con campos vacíos
Funcionalidad	RF1: Registrar nuevo producto en inventario.
Descripción	Verificar que el sistema no permita registrar un producto sin nombre.
Datos de Entrada	Nombre: vacío; Cantidad: 10; Precio compra: 0.50; Precio venta: 0.75.
Resultado Esperado	Mensaje de error indicando que el nombre es obligatorio.
Resultado Obtenido	El sistema muestra: El campo nombre es obligatorio.

Evidencia de ejecución:

CP1

```
CP1: Validacion de registro con campos vacios
Resultado esperado: El campo 'nombre' es obligatorio.
```

Captura de pantalla del caso CP1.

CP2: Registro correcto de producto perecedero

Elemento	Detalle
ID y Nombre	CP2: Registro correcto de producto perecedero
Funcionalidad	RF1: Registrar nuevo producto en inventario.
Descripción	Comprobar que se registre un producto perecedero con fecha de vencimiento válida.

Datos de Entrada	Producto: Leche; Cantidad: 20; Precio compra: 0.70; Precio venta: 1.00; Vence en 10 días.
Resultado Esperado	Producto registrado correctamente y listado con su tipo.
Resultado Obtenido	El sistema registra Leche como producto perecedero.

Evidencia de ejecución:

```
CP2
CP2: Registro correcto de producto perecedero
Producto registrado: ID:2 | Leche | Tipo:Perecedero (vence: 2026-06-11) | Cant:20 | PC:$0.70 | PV:$1.00 | Stock min:5
```

Captura de pantalla del caso CP2.

CP3: Listado y cálculo del valor total

Elemento	Detalle
ID y Nombre	CP3: Listado y cálculo del valor total
Funcionalidad	RF2: Consultar inventario.
Descripción	Verificar que el sistema liste productos y calcule el valor del inventario.
Datos de Entrada	Inventario con producto Leche registrado.
Resultado Esperado	Mostrar productos y valor total del inventario.
Resultado Obtenido	El sistema muestra el producto y valor total: \$14.00.

Evidencia de ejecución:

```
CP3
CP3: Listado y valor total del inventario
ID:2 | Leche | Tipo:Perecedero (vence: 2026-06-11) | Cant:20 | PC:$0.70 | PV:$1.00 | Stock min:5
Valor total: $14.00
```

Captura de pantalla del caso CP3.

CP4: Compra de producto

Elemento	Detalle
ID y Nombre	CP4: Compra de producto
Funcionalidad	RF5: Comprar producto.
Descripción	Verificar que la compra aumente el stock y actualice presupuesto.
Datos de Entrada	ID: 2; Unidades compradas: 10.
Resultado Esperado	Stock aumenta de 20 a 30 y el presupuesto disminuye.
Resultado Obtenido	Stock actual: 30; Presupuesto disponible: \$493.00.

Evidencia de ejecución:

```
CP4
CP4: Compra de producto
Compra registrada. Stock actual: 30
Presupuesto disponible: $493.00
```

Captura de pantalla del caso CP4.

CP5: Venta con stock insuficiente

Elemento	Detalle
ID y Nombre	CP5: Venta con stock insuficiente
Funcionalidad	RF6: Vender producto.
Descripción	Comprobar que el sistema impida vender más unidades que las disponibles.
Datos de Entrada	ID: 2; Unidades a vender: 100; Stock disponible: 30.
Resultado Esperado	Mensaje de error por stock insuficiente.
Resultado Obtenido	El sistema muestra: Stock insuficiente. Disponible: 30.

Evidencia de ejecución:

```
CP5
CP5: Venta con control de stock insuficiente
Resultado esperado: Stock insuficiente. Disponible: 30
```

Captura de pantalla del caso CP5.

CP6: Actualización de producto

Elemento	Detalle
ID y Nombre	CP6: Actualización de producto
Funcionalidad	RF3: Actualizar producto.
Descripción	Verificar que se modifiquen datos principales de un producto existente.
Datos de Entrada	ID: 2; Nuevo nombre: Leche entera; Precio compra: 0.72; Precio venta: 1.10; Stock mínimo: 8.
Resultado Esperado	Producto actualizado con los nuevos datos.
Resultado Obtenido	El sistema muestra el producto actualizado correctamente.

Evidencia de ejecución:

```
CP6
CP6: Actualizacion de producto
Producto actualizado: ID:2 | Leche entera | Tipo:Perecedero (vence: 2026-06-11) | Cant:30 | PC:$0.72 | PV:$1.10 | Stock min:8
```

Captura de pantalla del caso CP6.

CP7: Eliminación de producto

Elemento	Detalle
ID y Nombre	CP7: Eliminación de producto
Funcionalidad	RF4: Eliminar producto.
Descripción	Comprobar que el sistema elimine un producto mediante su ID.
Datos de Entrada	Producto temporal: Arroz; luego eliminar por ID.
Resultado Esperado	El producto debe eliminarse sin errores.
Resultado Obtenido	El sistema muestra: Producto eliminado correctamente.

Evidencia de ejecución:

```
CP7
CP7: Eliminacion de producto
Producto eliminado correctamente: Arroz
```


Captura de pantalla del caso CP7.

CP8: Alertas de bajo stock y vencimiento

Elemento	Detalle
ID y Nombre	CP8: Alertas de bajo stock y vencimiento
Funcionalidad	RF7: Alertas de inventario.
Descripción	Verificar que se detecten productos con bajo stock y perecederos vencidos.
Datos de Entrada	Producto: Yogurt; Cantidad: 3; Stock mínimo: 5; Fecha vencida.
Resultado Esperado	El producto debe aparecer en bajo stock y vencidos.
Resultado Obtenido	El sistema lista Yogurt en ambas alertas.

Evidencia de ejecución:

```
CP8
CP8: Alertas de bajo stock y vencimiento
Bajo stock:
ID:4 | Yogurt | Tipo:Perecedero (vence: 2026-05-31) | Cant:3 | PC:$0.60 | PV:$0.90 | Stock min:5
Vencidos:
ID:4 | Yogurt | Tipo:Perecedero (vence: 2026-05-31) | Cant:3 | PC:$0.60 | PV:$0.90 | Stock min:5
```

Captura de pantalla del caso CP8.

6. Conclusión

El proyecto propuesto resuelve el problema de inventario de una tienda pequeña mediante una solución funcional en Java. La alternativa seleccionada aplica encapsulamiento, herencia y polimorfismo, además de organizar el código en paquetes para mejorar la mantenibilidad. El sistema permite realizar el CRUD de productos, gestionar compras y ventas, controlar presupuesto, validar datos y generar alertas de bajo stock y vencimiento. Las pruebas ejecutadas evidencian que la solución cumple con los requerimientos planteados y se alinea con los criterios de la rúbrica: formulación integral del problema, alternativas viables y selección fundamentada técnicamente.